In [31]:
```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeature
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error
```

In [33]:
```
df = pd.read_csv("insurance.csv")
df.head(len(df))
```

Out[33]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
...	...	...	...	...	...	...	...
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

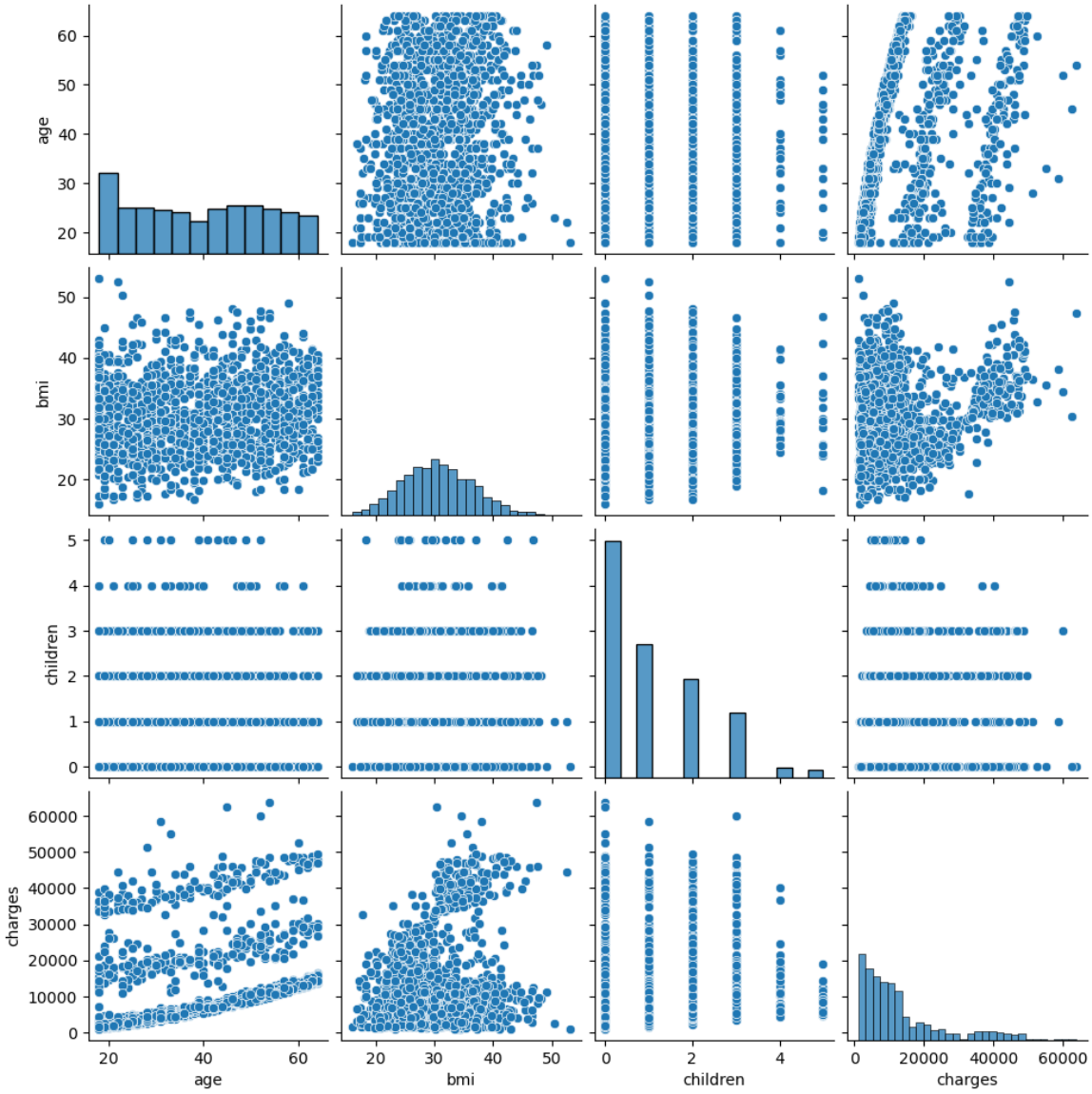
1338 rows × 7 columns

In [35]:
```
print(df.info())
print(df.describe())

sns.pairplot(df)
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None
```

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010



In [36]:
```
# Convert categorical variables using one-hot encoding
df = pd.get_dummies(df, drop_first=True)

df.head()
```

Out[36]:

	age	bmi	children	charges	sex_male	smoker_yes	region_north
0	19	27.900	0	16884.92400	False	True	
1	18	33.770	1	1725.55230	True	False	
2	28	33.000	3	4449.46200	True	False	
3	33	22.705	0	21984.47061	True	False	
4	32	28.880	0	3866.85520	True	False	

In [39]:
```
# Target variable
y = df['charges']

# Features (exclude target)
X = df.drop('charges', axis=1)

print(X.columns)
```

```
Index(['age', 'bmi', 'children', 'sex_male', 'smoker_yes', 'region_n
orthwest',
       'region_southeast', 'region_southwest'],
      dtype='object')
```

In [41]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [43]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [45]:
```
models = {}

# Linear Regression
models['Linear Regression'] = LinearRegression()
models['Linear Regression'].fit(X_train_scaled, y_train)

# Ridge Regression
models['Ridge Regression'] = Ridge(alpha=1.0)
models['Ridge Regression'].fit(X_train_scaled, y_train)

# Lasso Regression
models['Lasso Regression'] = Lasso(alpha=0.1)
models['Lasso Regression'].fit(X_train_scaled, y_train)

# Polynomial Regression (degree 2) → improves accuracy
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train_scaled)
```

```
X_test_poly = poly.transform(X_test_scaled)

poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)
models['Polynomial Regression'] = poly_model

# Random Forest (BEST for accuracy)
rf = RandomForestRegressor(n_estimators=300, max_depth=10, random_s
rf.fit(X_train, y_train)  # no scaling needed
models['Random Forest'] = rf
```

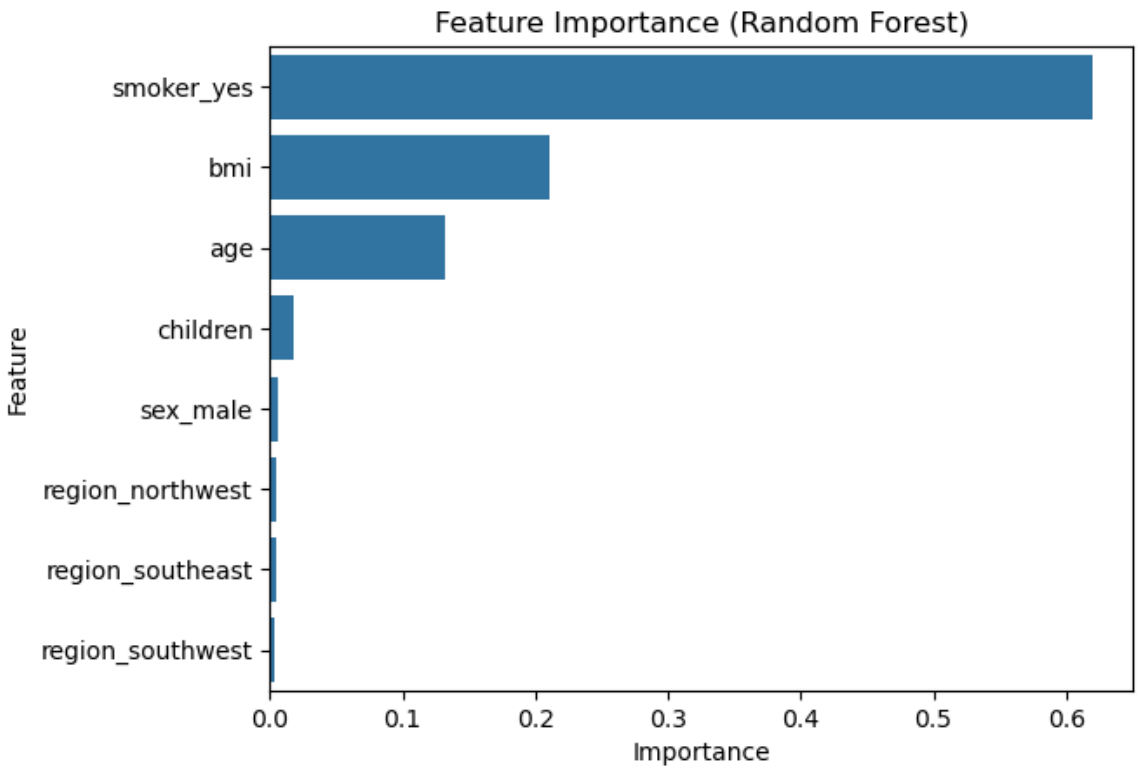
In [47]:

```
rf_model = models['Random Forest']

importances = rf_model.feature_importances_
features = X.columns

importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title("Feature Importance (Random Forest)")
plt.show()
```



In [49]:

```
results = []

for name, model in models.items():

    if name == 'Polynomial Regression':
        y_pred = model.predict(X_test_poly)
    elif name == 'Random Forest':
        y_pred = model.predict(X_test)
```

```
    else:
        y_pred = model.predict(X_test_scaled)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    results.append([name, mse, rmse, mae, r2])

# Convert to DataFrame
results_df = pd.DataFrame(results, columns=['Model', 'MSE', 'RMSE',

# Sort by best model
results_df = results_df.sort_values(by='R2', ascending=False)

results_df
```

Out[49]:

	Model	MSE	RMSE	MAE	R2
4	Random Forest	2.077884e+07	4558.380853	2520.556612	0.866158
3	Polynomial Regression	2.088336e+07	4569.831802	2864.034704	0.865484
0	Linear Regression	3.359692e+07	5796.284659	4181.194474	0.783593
2	Lasso Regression	3.359734e+07	5796.321036	4181.226051	0.783590
1	Ridge Regression	3.360497e+07	5796.979691	4182.796597	0.783541

In [51]:

```
for index, row in results_df.iterrows():
    print(f"\n🔷 {row['Model']} Performance")
    print(f"MSE  : {row['MSE']:.2f}")
    print(f"RMSE : {row['RMSE']:.2f}")
    print(f"MAE  : {row['MAE']:.2f}")
    print(f"R2   : {row['R2']:.4f}")
```

```
🔷 Random Forest Performance
MSE  : 20778836.00
RMSE : 4558.38
MAE  : 2520.56
R2   : 0.8662

🔷 Polynomial Regression Performance
MSE  : 20883362.70
RMSE : 4569.83
MAE  : 2864.03
R2   : 0.8655

🔷 Linear Regression Performance
MSE  : 33596915.85
RMSE : 5796.28
MAE  : 4181.19
R2   : 0.7836

🔷 Lasso Regression Performance
MSE  : 33597337.56
RMSE : 5796.32
MAE  : 4181.23
R2   : 0.7836

🔷 Ridge Regression Performance
MSE  : 33604973.54
RMSE : 5796.98
MAE  : 4182.80
R2   : 0.7835
```